

TeamMatch – Ethical, Legal, and Security Analysis

Name: Viet Nguyen

Project: TeamMatch

Ethical Issues

TeamMatch is designed to help instructors create balanced student teams based on skills and availability. Because the system uses student-provided data, privacy and fairness are important concerns.

Students provide information such as skills, availability, and confidence levels, which could be sensitive. The system should ensure that only necessary data is collected and that student information is not exposed to others unnecessarily. Instructors should only use this data for team formation and not for grading or ranking students.

Another ethical concern is fairness. If skill data is misused, students could be unfairly labeled as weaker or stronger contributors. To prevent this, TeamMatch uses the data only to balance teams and provide equal learning opportunities. The system avoids discrimination based on race, gender, religion, or other personal factors.

Legal Issues

TeamMatch must follow legal guidelines related to data privacy, intellectual property, and software licensing.

The system stores student-related data such as names, skills, and availability. This information should be handled carefully and used only for educational purposes. The system should not collect unnecessary personal data.

TeamMatch uses technologies such as Python, FastAPI, and JavaScript, which are open-source tools. These are legally allowed for use in this project. Any external tools or APIs should also follow proper licensing rules.

Users should not misuse the system to upload copyrighted content or share sensitive information. The system should restrict input and define acceptable use policies.

Security Issues

TeamMatch stores student data, which includes names, skills, and availability. This information must be protected from unauthorized access.

Possible security risks include unauthorized access, API misuse, and common web vulnerabilities such as Cross-Site Scripting (XSS) and injection attacks. Attackers could attempt to send malicious data to the API or access other users' information.

To prevent this, the system uses input validation to ensure all data is properly formatted. API endpoints only accept valid JSON requests. Sensitive data should be transmitted securely using HTTPS in a cloud environment.

The system should also implement authentication and authorization so that only authorized users can access or modify data. Best practices such as regular updates, secure coding, and proper error handling are used to reduce risks.